

SYNOPSIS

Title

FraudShield – Real-Time Credit Card Fraud Detection Gateway

Introduction

Digital payments have become the backbone of global commerce, and that growth has been matched by an equally rapid rise in sophisticated financial cybercrime. Credit card fraud costs businesses and consumers billions of dollars each year, and the real challenge for any detection system is not simply catching fraud, but catching it accurately and instantly, without disrupting genuine customers. FraudShield is a simulated payment gateway paired with a machine-learning-based fraud detection engine that evaluates transactions in real time and flags suspicious activity as it happens, moving beyond the rigid, manually-maintained rules that legacy systems rely on.

Problem Statement

Traditional fraud detection systems rely heavily on static, rule-based logic — flagging transactions that exceed a fixed amount, originate from an unusual location, or match other predefined patterns. This forces a difficult tradeoff: rules strict enough to catch fraud tend to generate excessive false positives, blocking legitimate purchases and frustrating customers, while rules loose enough to avoid false positives let novel or subtle fraud patterns slip through undetected. Because rule sets are manually defined, they also struggle to adapt as fraud tactics evolve. FraudShield addresses this by learning patterns directly from historical transaction data and scoring each transaction's fraud likelihood in real time, catching anomalies that static rules would miss while reducing friction for legitimate cardholders.

Objectives

- Simulate a realistic payment gateway capable of processing mock transactions end-to-end.
- Train and evaluate machine learning classifiers (Random Forest and SVM) to distinguish fraudulent from legitimate transactions.
- Minimize false positives while maintaining a high fraud-detection rate, rather than optimizing for raw accuracy alone.
- Score and flag transactions in real time as they are submitted, rather than through batch or offline review.
- Provide fraud analysts with a dashboard to review flagged transactions and feed corrected labels back into the system.

Scope

FraudShield is scoped as a simulated, academic-purpose payment gateway — it does not process real money, connect to real card networks, or handle genuine cardholder data, which keeps it outside real-world regulatory and compliance obligations. Transaction data is drawn from a labeled, anonymized public fraud-detection dataset, supplemented with synthetically generated transactions where needed, to train and evaluate the classification model. The system covers four user roles: Customers submitting simulated purchases, Merchants viewing transaction outcomes, Fraud Analysts reviewing flagged activity, and Admins managing detection thresholds and system settings. It does not cover integration with real banking infrastructure or card-network certification, which fall outside the scope of a solo academic project.

Proposed Solution / Methodology

FraudShield combines a rule-based pre-filter with a machine-learning classification layer. Historical transaction data — including anonymized features, transaction amount, and time — is preprocessed and used to train Random Forest and Support Vector Machine classifiers; because fraudulent transactions represent a small fraction of all

transactions, class-imbalance techniques such as SMOTE oversampling are applied during training, and models are evaluated using precision, recall, F1-score, and AUC-ROC rather than raw accuracy, since accuracy alone is misleading on imbalanced data. The best-performing model is serialized and loaded into the backend for real-time inference. When a simulated transaction is submitted, the system first applies lightweight rule-based checks, such as transaction velocity, then passes the transaction through the trained classifier, which returns a fraud-probability score. Transactions above a configurable threshold are automatically declined or routed to a fraud analyst for manual review; transactions below it are approved automatically. Analyst decisions on flagged transactions are logged and can be used to periodically retrain and improve the model.

Features/Modules

- Transaction Simulation Module – generate and submit mock transactions (amount, merchant, location, timestamp)
- Rule-Based Pre-Filter – lightweight checks such as transaction velocity and amount thresholds, applied before ML scoring
- Fraud Detection Engine – real-time ML-based classification (Random Forest / SVM) returning a fraud-probability score
- Merchant Dashboard – view transaction approval/decline outcomes
- Fraud Analyst Dashboard – review flagged transactions, approve or decline, and label outcomes
- Admin Panel – manage detection thresholds, user roles, and system configuration
- Model Performance Analytics – precision, recall, F1-score, and confusion-matrix visualization over time
- Notification System – real-time alerts to analysts when a high-risk transaction is flagged

Technology Stack

- Frontend: React, Tailwind CSS
- Backend: Django, Django REST Framework (Python)
- Machine Learning: scikit-learn (Random Forest, SVM), pandas, NumPy, imbalanced-learn (SMOTE)
- Database: PostgreSQL
- Model Serialization/Serving: joblib, served through Django REST endpoints
- Authentication: Django's built-in auth system, djangorestframework-simplejwt (role-based: Customer / Merchant / Analyst / Admin)
- Visualization: Recharts (analyst-facing performance dashboard)
- Hosting/Deployment: Vercel (frontend), Render (backend and database)

System Requirements

Hardware: Standard laptop or desktop with at least 8 GB RAM (16 GB recommended for comfortable model training and experimentation).

Software: Python 3.11+, Node.js 18+, PostgreSQL 15+, Git, and the scikit-learn / pandas / NumPy / imbalanced-learn ML stack.

Browser: Latest two versions of Chrome, Firefox, Edge, or Safari, with JavaScript enabled.

Operating System: Cross-platform — Windows, macOS, or Linux for development; end users need only a modern web browser.

Expected Outcomes

- A working simulated payment gateway that classifies transactions as fraudulent or legitimate in real time.
- A measurable improvement over naive rule-based detection, demonstrated through precision, recall, and F1-score on a held-out test set.

- A functional analyst dashboard that reduces the manual effort required to triage suspicious transactions.
- A clear demonstration of the precision/recall tradeoff inherent to fraud detection, and how it can be tuned via threshold selection.
- A solo-built, full-stack machine learning application demonstrating classification modeling, real-time inference, and role-based access control.

Future Scope

- A real-time streaming architecture (e.g., Kafka) to handle higher transaction throughput.
- Deep learning approaches, such as autoencoders for anomaly detection or graph-based models of transaction networks, to catch more subtle fraud patterns.
- Integration with a sandboxed real payment gateway (e.g., Stripe's test mode) for a more realistic simulation.
- Explainable AI techniques (e.g., SHAP values) so analysts can see why a given transaction was flagged.
- Continuous/online learning so the model adapts as new fraud patterns emerge, rather than relying solely on periodic retraining.

Conclusion

FraudShield tackles a costly, real-world problem — the tradeoff between catching fraud and disrupting legitimate customers — with an approach grounded in data rather than static rules. By combining a rule-based pre-filter, a trained classification model, and a real-time analyst review workflow, the project delivers a technically substantial build spanning data preprocessing, model training and evaluation, real-time inference, and role-based dashboards, while directly addressing a problem with clear, measurable business value — making it a strong fit for a solo capstone project.